

APUNTES TEMA 4

ÍNDICE:

1. Herencias
2. Polimorfismo
3. Interfaces

(Cap 40 - 55)

Se viene tipo test

10/12/2025

HERENCIAS

Las herencias en Java debemos comprender que solo vamos a poder heredar de una **generación anterior**, (*significa que no vas a poder heredar de los abuelos*)

CARACTERÍSTICAS:

- La subclase puede poseer atributos y métodos que no existan en la original.
- Se hereda **TODO** menos los **constructores**.

Para conectar las clases utilizaremos la siguiente **sintaxis**.

public class (nombre clase Hija) extends (nombre clase Madre) { }

MODIFICADOR FINAL

Sirve para indicar que es un final:

En el caso de las clases para indicar que es la última clase hija

En el caso de la escritura nos hará que no se pueda reescribir, ese sera el ultimo rescrito

Herramienta (funciona para asegurarse de que no habrá nada por debajo)

Ejemplo para poner final a una **clase**:

public final class (Clase hija) extends (Clase Madre) { }

Ejemplo para poner final a un **método**:

public final (tipo de dato) (nombre método) () { }

CLASES Y MÉTODOS ABSTRACTOS

CLASES

Cuando los objetos son demasiado **generales** y no se sepan definir ni instanciar denominaremos a estas clases como "**Abstractas**".

→ El punto es que no sabemos definirlo.

Estas clases nos prohíben instanciar los objetos de clases abstractas. Todos los objetos que instanciamos deben ser de las clases de las **hijas**.

Se pueden tener métodos en la clase abstracta?

→ Se crearán **métodos abstractos**: se creará el método de forma **abstracta** y sin agregar un cuerpo (ya que desconocemos su función)

- (Una clase abstracta **no** puede ser **final**)
- (Para que sea una clase abstracta debe tener mínimo un método abstracto)

¿CÓMO INDICAR QUE UNA CLASE ES ABSTRACTA?

→ `public abstract class (clase madre) { }`

FILOSOFÍA → Los programas deben seguir la filosofía de que contra más generales sean las cosas mejor (ya que rendirá mejor en el futuro).

¿HACE FALTA CREAR CONSTRUCTORES EN LAS CLASES ABSTRACTA?

→ Si, ya que es necesario para crear **los objetos** de la clase hija.

Para que una clase sea abstracta → Es obligatorio que **al menos tenga un método abstracto**.

CARACTERÍSTICA IMPORTANTE CLASES ABSTRACTAS

Es importante comprender que al tener un método de una clase abstracta estamos obligados a **REESCRIBIR** en una clase hija. Y si no queremos reescribir el método estamos **obligados** a hacer esa clase hija también de forma abstracta.

El **override** nos indica que es un método **reescrito**.

INTERFACES (Clases con todos los métodos abstractos)

No se pueden crear objetos de una clase abstracta pero sí se pueden crear **arrays** de una clase abstracta.

ESTÁTICOS

